WEIR - Assignment 1 - Preferential web crawling

Maj Bregar, Gal Fajon, Alen Leban
Faculty of Computer and Information Science
Večna pot 113, 1000 Ljubljana

***Abstract*—In this assignment we developed a preferential web crawler for the news site 24ur.com[1] that collects pages relevant to a user-specified topical query (for example, the conflict between Russia and Ukraine). Relevance is estimated by embedding page metadata, such as titles, summaries, and keywords, into a vector space using a pre-trained language model, then computing cosine similarity between the page embeddings and an embedding of the query. The crawler is comprised of an SQL database server, a RESTful API server that exposes database operations, and a client that uses browser automation to extract page content and hyperlinks and applies machine-learning scoring to prioritize crawl targets.**

I. Introduction

This report describes the design and implementation of a preferential web crawler developed to extract pages relevant to a user-specified topical query within a chosen domain. For this assignment, the chosen domain is the Slovenian news website 24ur.com.

The crawler comprises several components: a headless browser HTTP downloader and renderer, a data extractor for hyperlinks and images, a duplicate detector, a preferential URL frontier and a PostgreSQL database accessed via a RESTful API. The system is designed to run multiple parallel workers.

Pages are prioritised according to their relevance to a user-specified topical query (e.g. the conflict between Russia and Ukraine) by embedding page metadata into a natural-language string that is then converted into a vector representation using a pre-trained language model and ranked using cosine similarity.

The implementation also enforces canonical URLs, deduplication, respects robots.txt directives and crawl delay constraints and records pages, images and binary files found within the domain in the database.

The following sections document implementation details, parameter choices, performance evaluation, and visualisations of the resulting data.

II. Methodology

A. Crawler Architecture

The crawler is implemented as a modular system consisting of a RESTful API, a database server, and a client service. Is is deployed using a Docker Compose project, with service configuration and crawling parameters defined in the Docker Compose project configuration file.

B. API Server and Database

The RESTful API service is implemented using a Python Flask[**?**] web server. It utilizes the SQLAlchemy[**?**] library to define and initialize the database schema in accordance with the schema provided in the assignment instructions.

The server supports configurable HTTP Basic authentication and uses Flask's built-in ad hoc SSL context to enable HTTPS, ensuring secure communication between components.

The crawler client interacts with the API using a Python module, which sends HTTP requests using the requests[**?**] library.

C. Crawler client

The crawler client uses the threading module to spawn a specified number of worker threads, which process URLs from a shared URL frontier. The Selenium [2] Firefox headless web browser is used to parse pages and extract metadata.

Links found on pages are extracted, scored, and added to the frontier. This process repeats until the desired number of pages has been crawled.

If an insertion into the database fails due to URL or content hash uniqueness constraints, the client handles the error by creating a page entry, marked as a duplicate.

The crawler respects the `robots.txt` file of the specified domain by adhering to the specified crawl delay.

D. Page parsing

Each subpage is parsed using the BeautifulSoup[**?**] module. The parser is used to extract article keywords and link URLs to other pages (in the `href` property or `onclick` JavaScript event handler).

The `onclick` event handler is processed by extracting URL candidates with regular-expression patterns that detect common JavaScript redirects as well as embedded JSON-like fields (`Url:`, `PlaybackUrl:`); if none match, it falls back to any quoted `.html` substring and returns the list of candidate URLs.

When saving a page to the database, each extracted URL is canonicalized using urllib[**?**] and tldextract[**?**]. Redirection links belonging to the domain `go-ctr-tracker.pub.24ur.si` are resolved using the `redir` key in the query string, the host is reduced to the highest level domain, the path is normalized, removing trailing slashes and redundant separators, common tracking parameters relating to UTM, as well as `fbclid`, `gclid`, `yclid` are removed, remaining query parameters are sorted

deterministically, and the final URL is returned with the `https` scheme and without fragments.

Binary resources are detected by file extension (`.pdf`, `.doc`, `.docx`, `.ppt`, `.pptx`). For HTML pages, all outgoing `<a href>` targets are canonicalized and then checked for these extensions; matching targets are not added to the link frontier, but are instead stored as page data entries with their type code set to the corresponding type.

Images are detected in a similar way, using common image file extensions (`.jpg`, `.jpeg`, `.png`, `.gif`, `.webp`, `.bmp`, `.svg`), and are stored in the corresponding database table.

E. Preferential Crawling

Preferential crawling is implemented by maintaining a shared in-memory URL frontier as a priority queue. Each worker repeatedly removes the currently best-ranked URL and processes the corresponding page.

For each discovered URL, the crawler first applies canonicalization and filters out already visited URLs, off-domain URLs, and URLs disallowed by `robots.txt`. The parsing stage associates each link with metadata such as an inferred link title (from the URL slug), an optional summary (from nearby `p`/`span` elements with class `summary`), page-level article keywords (from the `article-keywords` block), and additional keyword-like text (`span` elements with the `label` class).

Since the same URL can be encountered multiple times from different pages and page regions, metadata is accumulated across occurrences in a per-URL list; every time new metadata is appended, a per-URL *version counter* is incremented.

Relevance scoring is performed with a multilingual sentence-transformer model (`paraphrase-multilingual-MiniLM-L12-v2` [3]). At the start of crawling the user query is embedded once, producing a normalized query embedding $\mathbf{e}_q$. For every candidate frontier URL u, the crawler constructs a short natural-language metadata string by concatenating de-duplicated fields in the form *Naslovi:* … *Povzetki:* … *Ključne besede:* … and embeds it into $\mathbf{e}_u$. The relevance score is then computed as cosine similarity

$$s(u) = \cos(\mathbf{e}_q, \mathbf{e}_u). \tag{1}$$

Candidates with no available metadata are assigned an empty metadata string and a neutral score of 0. Because the priority queue dequeues the smallest numeric key, the crawler converts similarity into a frontier priority by negation ($p(u) = -s(u)$), causing more relevant pages (larger s) to be scheduled earlier. To avoid using outdated scores, frontier entries carry the version number that was current when they were inserted; if a URL is encountered again and its metadata (and thus its version) increases, older queued entries are discarded when popped.

In addition to the in-memory frontier, every candidate is also stored in the database as a frontier page with its current priority, and a link edge is recorded from the currently crawled page to the corresponding frontier entry; when a frontier URL is later crawled, that database entry is updated with the full page content.

F. Deduplication

To avoid storing identical (or near-identical) content multiple times, every downloaded HTML page is assigned a content hash and the database enforces uniqueness of this hash. The hash is computed with the ppdeep[**?**] library.

Before hashing, the HTML is parsed and non-content elements (`script`, `style`, `noscript`) are removed. If the page contains an `<article>` element, its text is used as the primary content; two highly dynamic sub-blocks are explicitly removed when present: the comments placeholder (`id=comments_placeholder`) and a promotional widget (`data-upscore-zone=microsite_promo`). The remaining article text is extracted, whitespace-normalized, and hashed. For non-article pages, a fallback signature is constructed from stable navigation/text cues, such as submenu item titles and page headers (e.g., `h1` elements under `pageContent`).

The resulting hash is stored in the content hash field of the page table, which is protected by a unique constraint. When inserting or updating a page violates this constraint, the client resolves the conflict by locating the already-stored page with the same hash and creating a new entry marked as a duplicate (with the content cleared and the hash omitted to avoid retriggering the constraint). A link is then recorded from the duplicate entry to the original page, preserving the crawl graph while preventing redundant storage of page bodies.

III. Crawling Analysis

We analyzed the performance of our crawler on only one site, as mentioned above. When analyzing the results of our crawling, we discovered the following statistics:

Page count	Duplicate count	Image count	Avg. images per page
39874	13	121238	3

A. Crawling Quality

To assess the quality of the crawler, we performed a semi-manual evaluation by constructing an interactive data map in which HTML pages are represented as two-dimensional points. This visualization enables inspection of the semantic structure of the collected data and helps identify potential inconsistencies or noise in the crawling process.

The pipeline begins by encoding page content using TF-IDF with a vocabulary size of 15,000. To reduce the impact of high-frequency noise and compress the representation, the vectors are projected into a 40-dimensional space using truncated Singular Value Decomposition (SVD). Clustering is then performed on these reduced representations using the K-means algorithm, where the optimal number of clusters is selected based on the silhouette score.

For visualization, the clustered data is embedded into two dimensions using UMAP, which preserves local and global

structure sufficiently for qualitative inspection. To improve interpretability, each cluster is labeled with representative keywords. This is achieved by concatenating the text of all documents within a cluster into a single corpus, applying TF-IDF to extract 30 candidate keywords, and then re-ranking them using a BERT-based semantic similarity model. The top six keywords are selected to describe each cluster.

B. 24Ur.com Topography

- visualization of link table - maybe vis representing clusters as nodes, size of connection between nodes how many links are between those clusters?

IV. Disscussion

- mention the fact we do not pick up many duplicates/binary files - mention crawling quality, how we could improve it etc

V. Conclusion

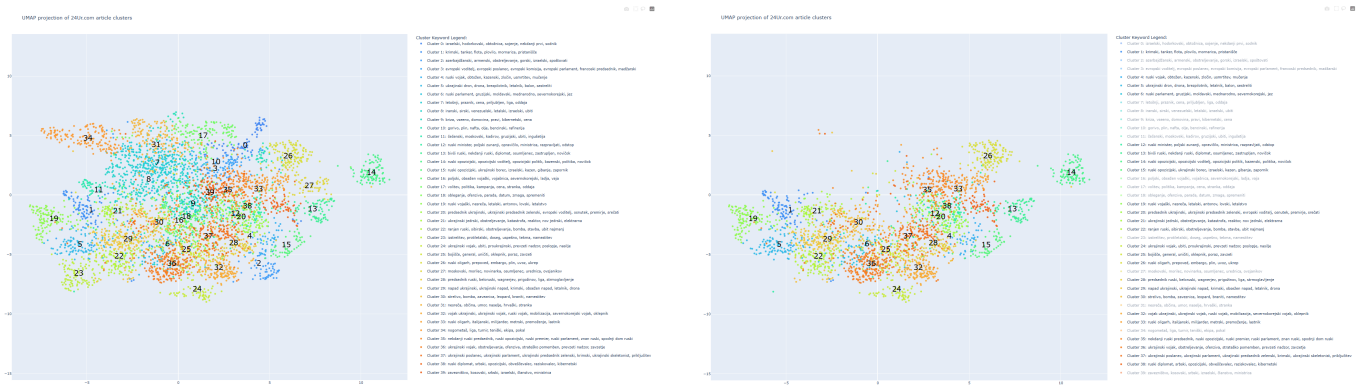
- yap yap yap

References

[1] 24ur. 24ur website. [Online]. Available: https://www.24ur.com/

[2] Selenium. [Online]. Available: https://www.selenium.dev/

[3] Huggingface. Bert model. [Online]. Available: https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2



(a) All clusters

(b) Query clusters

Fig. 1: UMAP projections of crawled HTML pages. (a) All pages grouped into semantic clusters. (b) Query-specific subset highlighting cluster distribution.